

TD n° 2 : Automates finis

Objectifs : concevoir et manipuler des automates finis,
explorer la correspondance entre expressions régulières et automates.

Exercice 1 — Conception d'automates finis

Pour chacun des langages, proposer un automate fini (à représenter sous forme graphique), qui le reconnaisse.

1. $\mathcal{L}_1 = \{ m \in \Sigma^* \mid \text{chaque position impaire contient un } 1 \}$, $\Sigma = \{0, 1\}$, $\varepsilon \notin \mathcal{L}_1$. NB : la numérotation des positions commence à 1.

Réflexions :

- a. La position 1 correspond forcément à une transition qui part de l'état initial.
- b. Inversement, si on veut discriminer les positions impaires et les positions paires, il faut aussi discriminer les états, ceux qui permettent des transitions de type impair, et ceux qui permettent des transitions de type pair.
- c. L'état initial est donc de type impair, mais il ne doit pas accepter le mot vide. Il faut donc 2 états de type impair : l'un qui accepte le mot vide (pour toutes les positions impaires, sauf la 1), l'autre qui ne l'accepte pas (pour la position 1).
- d. On a donc au moins 3 états : $(I)MPAIR_{\neq \varepsilon}$, $(I)MPAIR_{\varepsilon}$ et $(P)AIR$.
- e. On a obligatoirement $Q_F = \{ I_{\varepsilon}, P \}$ pour accepter des mots de longueurs paires ou impaires, et $q_0 = I_{\neq \varepsilon}$ pour ne pas accepter le mot vide.
- f. Et donc l'automate suivant :

$$M_1 = \{ \Sigma, \{ I_{\neq \varepsilon}, I_{\varepsilon}, P \}, q_0, q_F, \\ \{ (I_{\neq \varepsilon}, 1, P), (I_{\varepsilon}, 1, P), (P, 0, I_{\varepsilon}), (P, 1, I_{\varepsilon}) \} \}$$

qu'il vous revient de dessiner.

Questions :

- a. Modifier l'automate pour accepter le mot vide.
- b. Imaginer une structure de donnée informatique qui permette de représenter n'importe quelle machine M , et donc M_1 , dans le langage de programmation de votre choix.
- c. Un automate qui distingue entre positions paires et positions impaires peut-il posséder des transitions qui vont d'un état vers le même état (ex. (E, x, E)) ?

Remarques :

- a. Un automate qui réalise une consigne donnée a priori doit raconter une histoire qui rappelle la consigne. C'est pourquoi il est fort déconseillé de

nommer les états 0, 1, 2, 3, ... Il vaut mieux leur donner des noms qui rappellent les consignes, ici $\text{IMPAIR}_{\neq \varepsilon}$, $\text{IMPAIR}_{\varepsilon}$ et PAIR .

- b. Un état détermine comment réagir aux lettres lues. Si on envisage plusieurs schémas de réaction, on doit avoir au moins autant d'états.

2. $\mathcal{L}_2 = \{ m \in \Sigma^* \mid m \text{ commence par une lettre majuscule et se termine par un ou plusieurs chiffres} \}$, $\Sigma = \{a..z, A..Z, '-', 0..9\}$, $\varepsilon \notin \mathcal{L}_2$.

Réflexions :

- a. Commencer par une majuscule : l'état initial n'admet que des transitions étiquetées par une majuscule.
- b. Terminer par un chiffre : les états finaux ne sont accessibles que par des transitions étiquetées par un chiffre.
- c. Terminer par un ou plusieurs chiffres : on peut boucler sur les états finaux par des transitions étiquetées par un chiffre.
- d. Conclusion : on définit un état initial $q_0 = (\text{S})\text{TART}$, un ensemble, singleton, d'états finaux, $Q_F = \{ (\text{E})\text{ND} \}$, un état intermédiaire $(\text{B})\text{ODY}$, $Q = \{ S, B, E \}$, et l'automate suivant,
 $M_2 = \{ \Sigma, Q, q_0, Q_F, \{ (S, A..Z, B), (B, \Sigma, B), (B, 0..9, E), (E, \Sigma, B), (E, 0..9, E) \} \}$

Questions :

- a. Où est-il écrit que les triplets $(from, letter, to)$ entretenaient la dépendance fonctionnelle $(from, letter) \rightarrow to$?
- b. Essayer de produire par la réflexion un automate équivalent, mais qui vérifie cette dépendance fonctionnelle.
- c. Donner une variante où la consigne demanderait que chaque mot se termine par 2 chiffres ou plus.

Remarques :

- a. Se rappeler qu'un état final est une autorisation d'arrêter la lecture, mais pas une obligation de l'arrêter. Accepter un mot est alors la propriété que la fin du mot coïncide avec un état final. C'est pourquoi on les appelle aussi parfois « états d'acceptation ».
- b. On a fait un abus de notation en étiquetant les transitions par des ensembles de lettre. Il faut juste y voir une notation compacte d'un ensemble de transitions.

3. $\mathcal{L}_3 = \{ m \in \Sigma^* \mid m \text{ ne contient pas 2 '-' consécutifs et commence par une lettre} \}$, $\Sigma = \{a..z, '-', 0..9\}$, $\varepsilon \notin \mathcal{L}_3$.

Réflexions :

- a. La problématique nouvelle est de permettre des '-' tout en interdisant qu'ils se suivent.

- b. Il faut absolument pouvoir distinguer le fait de venir de lire la lettre '—' du fait de venir de lire autre chose. D'où un état HASREAD— et un état HAS-NOTREAD—.
- c. Entre ces deux états on aura les transitions suivantes :
 - $(\text{HR—}, \Sigma \setminus \{—\}, \text{HNR—})$
 - $(\text{HNR—}, \Sigma \setminus \{—\}, \text{HNR—})$
 - $(\text{HNR—}, '—', \text{HR—})$
- d. D'où l'automate final :

$$M_3 = \{ \Sigma, \{ (S)\text{TART}, \text{HR—}, \text{HNR—} \}, S, \{ \text{HR—}, \text{HNR—} \}, \\ \{ (S, a..z, \text{HNR—}), \\ (\text{HNR—}, \Sigma \setminus \{—\}, \text{HNR—}), (\text{HNR—}, '—', \text{HR—}), \\ (\text{HR—}, \Sigma \setminus \{—\}, \text{HNR—}) \} \}$$

Questions :

- a. Où est-il écrit que les mots de $\mathcal{L}_3$ doivent contenir le caractère '—' ?
- b. Proposer une variante de $\mathcal{L}_3$ où les mots doivent aussi contenir le caractère '—'.

Remarques :

- a. Comme pour les expressions régulières (et les bases de données), les consignes négatives sont souvent plus fastidieuses à modéliser que les consignes positives.
- b. Il est souvent de bonne stratégie d'isoler un trait de langage pour le résoudre, puis le fondre avec les autres traits. Comme pour les expressions régulières, mais aussi les requêtes de bases de données, il est souvent illusoire de produire une solution en un seul coup.

4. $\mathcal{L}_4 = \{ m \in \{a,b\}^* \mid |m| \text{ paire} \}.$

5. $\mathcal{L}_5 = \{ m \in \{a,b\}^* \mid |m| \text{ impaire} \}.$

Réflexions :

- a. Il s'agit évidemment de 2 fois la même question, mais habillée différemment.
- b. On peut réutiliser l'idée des états PAIR et IMPAIR de la question 1.
- c. On reconnaîtra un langage de mots de longueur impaire en terminant la lecture sur l'état PAIR (resp. longueur paire et état IMPAIR_ε ou IMPAIR_{≠ε}).
- d. Vu comme cela l'état initial est IMPAIR_ε. On peut en faire un état final pour le langage des mots de longueur paire si le mot ε est accepté. Sinon, ce sera IMPAIR_{≠ε} qui sera état final.

Questions :

- a. Finaliser les réponses aux questions 4 et 5.

Remarques :

- Un état est caractérisé par l'ensemble des mots qui permettent de l'atteindre. Mais il caractérise aussi les transitions autorisées à la suite de la lecture d'un de ces mots.
- Les langages intéressants contiennent une infinité de mots, donc une infinité de sous-ensembles. Cependant, le nombre d'états d'un automate fini est fini par définition. Cela signifie que les transitions permises sont déterminées par un nombre fini de ces sous-ensembles. C'est ça la magie des langages rationnels.

Exercice 2 — Déterminisation d'automates finis

Pour chacun des 2 automates suivants, indiquer la cause de son non-déterminisme, le déterminer et indiquer le langage qu'il reconnaît.

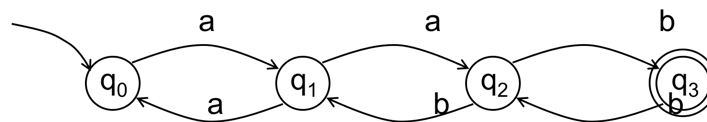


Figure 1: Automate reconnaissant le langage $\mathcal{L}_6$

Réflexions :

- L'état q_1 ne vérifie pas la dépendance fonctionnelle (*from, letter*) $\rightarrow$ *to*, car il permet une transition (q_1, a, q_0) et une transition (q_1, a, q_2) . L'état q_2 est similaire. On dit que cet automate n'est pas « déterministe » ou qu'il est « indéterministe ».
- La théorie nous dit que tout langage rationnel peut être reconnu par un automate déterministe. La démonstration de ce théorème se fait en montrant comment dériver d'un automate non-déterministe un automate qui reconnaît le même langage. On appelle ce calcul la « déterminisation » de l'automate. L'automate dérivé a le même vocabulaire que l'automate de départ, mais ses états sont des ensembles d'états de l'automate de départ.
- Pour déterminer un automate on construit ligne après ligne une table dont le schéma est le suivant :

FROM	σ_1	σ_2	...	σ_n
$\{q_0\}$	$\{q_i \mid \exists q \in \text{FROM. } q \rightarrow^{\sigma_1} q_i\}$	$\{q_i \mid \exists q \in \text{FROM. } q \rightarrow^{\sigma_2} q_i\}$		$\{q_i \mid \exists q \in \text{FROM. } q \rightarrow^{\sigma_n} q_i\}$
...	...	...	...	...

où

- FROM contient des ensembles d'états de l'automate à déterminer. La première ligne de FROM est initialisée au singleton état initial. Les suivantes ne peuvent prendre que des valeurs

d'ensembles d'états lus dans les colonnes σ_i d'une ligne précédente.

- Les colonnes σ_i (une par élément du vocabulaire) contiennent des ensembles d'états qui sont états d'arrivée d'une transition par la lettre σ_i en partant de l'état de la case FROM.
- On arrête la construction quand tous les ensembles d'états construits dans les colonnes σ_i sont traités dans la colonne FROM d'une ligne.
- Les ensembles de la colonne FROM sont les états de l'automate déterminisé. Chaque ligne montre alors les transitions permises à partir de chaque état de l'automate déterminisé. Les cases qui contiennent l'ensemble vide correspondent à des transitions interdites. Le singleton $\{q_0\}$ est l'état initial de l'automate déterminisé. Tous les ensembles qui contiennent un état final de l'automate de départ sont des états finaux de l'automate déterminisé.

d. Application à l'automate de la Figure 1 :

FROM	a	b
$\{q_0\}$	$\{q_1\}$	$\{\}$
$\{q_1\}$	$\{q_0, q_2\}$	$\{\}$
$\{q_0, q_2\}$	$\{q_1\}$	$\{q_1, q_3\}$
$\{q_1, q_3\}$	$\{q_0, q_2\}$	$\{q_2\}$
$\{q_2\}$	$\{\}$	$\{q_1, q_3\}$

où on observe que l'état déterministe $\{q_1, q_3\}$ est final pour l'automate déterminisé, puisque q_3 est final pour l'automate de départ.

Questions :

- Pour quelle raison peut-on préférer qu'un automate soit déterministe ?
- Combien peut-il y avoir d'états dans l'automate déterminisé.
- Refaire l'exercice pour l'automate de la Figure 2.

Remarques :

- Ne pas être déterministe n'est pas forcément mal. C'est même très commode pour la modélisation (voir question 2).
- Bien réfléchir aux relations entre les ensembles suivants : langages rationnels, automates finis, automates finis déterministes.

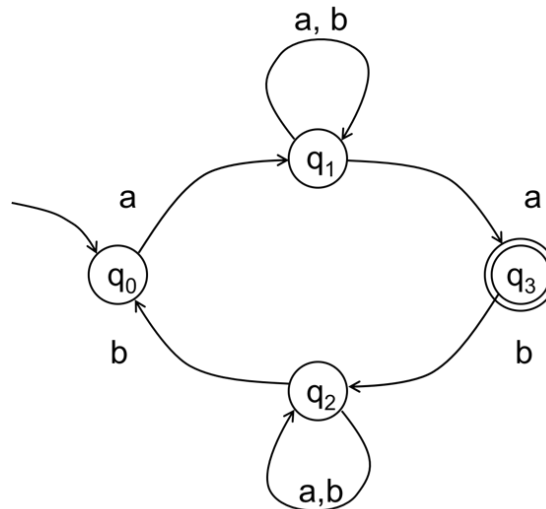


Figure 2 : Automate reconnaissant le langage $\mathcal{L}_7$

Exercice 3 — Automates finis et expressions régulières

Pour chaque description, dessiner l'automate et trouver une expression régulière dénotant le langage qu'il reconnaît.

1. $\Sigma = \{a, b\}$,
 $Q = \{q_1, q_2\}$, $F = \{q_1\}$,
 $\delta = \{ (q_1, a, q_1), (q_1, b, q_2), (q_2, a, q_2), (q_2, b, q_1) \}$,
 $M = \{ \Sigma, Q, q_2, F, \delta \}$.

Réflexions :

- a. On voit que lire la lettre 'b' fait passer de q_1 à q_2 puis de q_2 à q_1 . À chaque fois qu'un 'b' est lu la machine change d'état.
- b. Inversement, lire la lettre 'a' ne fait pas changer d'état.
- c. La machine lit donc les 'a' comme un bruit de fond, et lit les 'b' en en retenant la parité. L'état q_1 correspond à avoir lu 1 'b', 3 'b', ..., un nombre impair de 'b'. L'état q_2 correspond à avoir lu 0 'b', 2 'b', ..., un nombre pair de 'b'.
- d. Et comme le seul état final est q_1 , M reconnaît le langage des mots sur 'a' et 'b' qui contiennent un nombre quelconque de 'a' et un nombre impair de 'b'.
- e. Voir le TD 1 pour une traduction en expression régulière.

Questions :

- a. Produire l'expression régulière.
- b. On dit souvent que les automates finis ne savent pas compter. Proposer une variante un peu plus précise de cette affirmation.

Remarques :

- a. Un automate raconte une histoire. Il faut absolument s'entraîner à lire les histoires racontées par les automates.
- b. Souvent on lit plus facilement l'histoire racontée par un automate dans sa représentation graphique. Mais attention, on peut produire des automates qui par leur taille défient la représentation graphique. Ils ont en général été produits automatiquement.

2. $\Sigma = \{a, b\}$,
 $Q = \{q_0, q_1, q_2\}$, $F = \{q_2\}$,
 $\delta = \{ (q_0, a, q_0), (q_0, b, q_1), (q_1, a, q_0), (q_1, b, q_2), (q_2, a, q_2), (q_2, b, q_2) \}$,
 $M = \{\Sigma, Q, q_0, F, \delta\}$.

Réflexions :

- a. On voit que les trois états peuvent tous lire 'a' et 'b', mais ils n'en font pas la même chose.
- b. L'état q_2 permet de lire un nombre indéterminé de 'a' et de 'b', sans aucune contrainte. Et comme il est un état final, il accepte ces mots. En fait, q_2 permet de lire $(a | b)^*$.
- c. L'état q_0 permet de lire un nombre quelconque de 'a' puis un 'b'. En fait, q_0 permet de lire des mots qui commencent par a^*b .
- d. L'état q_1 permet de lire un 'a' en transitant vers q_0 ou de lire un 'b' en transitant vers q_2 .
- e. La transition de q_0 à q_2 se fait donc en lisant 2 'b' de suite.
- f. Le langage reconnu par cet automate est donc décrit par

$$RE = (a | ba)^* b b (a | b)^* .$$

Questions :

- a. Vous trouvez ça simple ?

Remarques :

- a. Dans les deux questions précédentes la traduction d'un automate vers une expression régulière passe par la compréhension du langage reconnu par l'automate. C'est beaucoup demander dès que l'automate n'est même que modérément complexe. On préférerait une méthode qui permette de produire une expression régulière sans même comprendre quel langage reconnaît l'automate. C'est l'objet de l'exercice 4.

Exercice 4 — Traduction d'automates finis en expressions régulières

Pour chacun des 2 automates suivants, trouver une expression régulière dénotant le langage qu'il reconnaît.

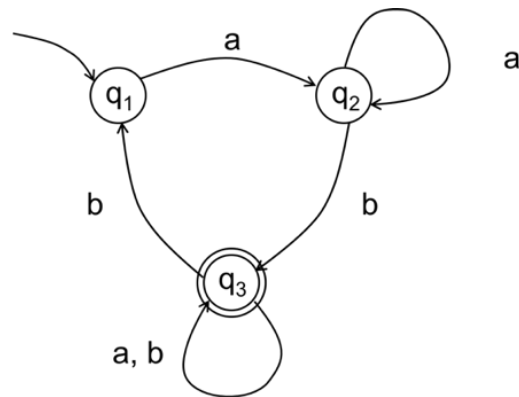


Figure 3 : Automate reconnaissant le langage $\mathcal{L}_8$

Réflexions :

- La clé d'une méthode automatique de traduction d'un automate en une expression régulière équivalente est de lire un automate comme un système d'équations dont les variables sont des langages.
- Se rappeler qu'un automate accepte les mots qui étiquètent tous les chemins qui vont de l'état initial à un état final. C'est un langage et on va l'appeler L_1 pour l'automate de la figure 3.
- Appelons L_2 le langage de tous les mots qui étiquètent les chemins qui vont de q_2 à un état terminal, et L_3 le langage de tous les mots qui étiquètent les chemins qui vont de q_3 à un état terminal. Que peut-on dire de L_1 , L_2 et L_3 ?
- On peut relier les 3 langages par les équations suivantes :

$$L_1 = a \bullet L_2$$

$$L_2 = a^* \bullet b \bullet L_3$$

$$L_3 = (a \mid b)^* \bullet (b \bullet L_1 \mid \varepsilon) = (a \mid b)^* \bullet b \bullet L_1 \mid (a \mid b)^* .$$
- Il ne reste plus qu'à résoudre ce système d'équation, et le lemme de Arden va nous y aider. Le lemme de Arden dit que la solution la plus générale d'une équation $X = A \bullet X \mid B$ est $X = A^* \bullet B$ (resp. $X = B \bullet A^*$ pour résoudre l'équation $X = X \bullet A \mid B$).
- Il suffit de faire des substitutions de variable et d'appliquer le lemme de Arden pour résoudre ce système d'équations :

$$L_1 = a \bullet L_2 = a \bullet a^* \bullet b \bullet L_3$$

$$= a \bullet a^* \bullet b \bullet ((a \mid b)^* \bullet b \bullet L_1 \mid (a \mid b)^*)$$

$$= a^+ \bullet b \bullet ((a \mid b)^* \bullet b \bullet L_1 \mid (a \mid b)^*)$$

$$= a^+ \bullet b \bullet (a \mid b)^* \bullet b \bullet L_1 \mid a^+ \bullet b \bullet (a \mid b)^*$$
- D'où $L_1 = (a^+ \bullet b \bullet (a \mid b)^* \bullet b)^* \bullet a^+ \bullet b \bullet (a \mid b)^*$

Questions :

- Considérons une définition récursive de la propriété « être une puissance de 2 » : $P = 2 \times P$ ou $P = 1$. Comparer avec le Lemme de Arden.
- Pourquoi le lemme de Arden a-t-il deux versions ?

Remarques :

- a. La traduction est devenue complètement mécanique. Elle ne travaille que sur les symboles et par sur le sens global pris par leur assemblage. C'est ce qu'on aime en informatique !

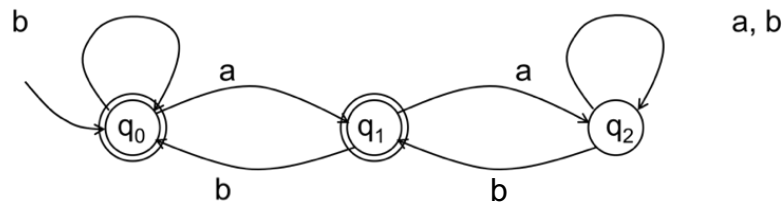


Figure 4 : Automate reconnaissant le langage $\mathcal{L}_9$

Réflexions :

- a. On applique la méthode automatique vue ci-dessus :
- $$\begin{aligned}
 L_0 &= b \bullet L_0 \mid a \bullet L_1 \\
 L_1 &= b \bullet L_0 \mid a \bullet L_2 \\
 L_2 &= b \bullet (L_1 \mid L_2) \mid a \bullet L_2 \\
 &= b \bullet L_1 \mid (a \mid b) \bullet L_2
 \end{aligned}$$
- b. On applique le Lemme de Arden et quelques réarrangements et substitutions de variable :
- $$\begin{aligned}
 L_2 &= (a \mid b)^* \bullet b \bullet L_1 && \text{par Arden} \\
 L_1 &= b \bullet L_0 \mid a \bullet (a \mid b)^* \bullet b \bullet L_1 && \text{par substitution} \\
 L_1 &= (a \bullet (a \mid b)^* \bullet b)^* \bullet b \bullet L_0 && \text{par Arden} \\
 L_0 &= b \bullet L_0 \mid a \bullet (a \bullet (a \mid b)^* \bullet b)^* \bullet b \bullet L_0 && \text{par substitution} \\
 L_0 &= (b \mid a \bullet (a \bullet (a \mid b)^* \bullet b)^* \bullet b) \bullet L_0 && \text{par réarrangement} \\
 L_0 &= (b \mid a \bullet (a \bullet (a \mid b)^* \bullet b)^* \bullet b)^* && \text{par Arden}
 \end{aligned}$$
- c. Et voilà !

Questions :

- a. Qu'est-ce qui autorise les réarrangements de l'expression du langage L_2 ?
- b. Qu'est-ce qui autorise les réarrangements de l'expression du langage L_0 ?

Remarques :

- a. On procède ici comme pour la résolution à la main d'autres systèmes d'équations.
- b. On applique des lemmes spécifiques au domaine de calcul (ici le lemme de Arden) qui permettent d'éliminer des variables.
- c. On applique des transformations générales qui sont justifiées par les axiomes du domaine de calcul (élément neutre, commutativité, distributivité, associativité, etc.).

- d. En général, on applique c. avec une idée derrière la tête qui est de faciliter l'application de b. C'est tout l'art de la résolution d'équation !
- e. Quand la réponse attendue est juste la valeur d'une des variables il est plus direct de finir par elle. Du coup, les variables qui sont dans la même équation deviennent des réponses attendues et ainsi de suite. Cela donne une idée de l'ordre dans lequel faire les choses.

Exercice 5 — Traduction d'expressions régulières en automates finis

À partir de chaque expression régulière RE_i trouver un automate reconnaissant $\mathcal{L}(RE_i)$.

1. $RE_1 = (ab \mid a^+)^*$
2. $RE_2 = (a \mid b)^* aab$

Réflexions :

- a. Le théorème de Kleene (en fait un des nombreux théorèmes de Kleene) dit que si un langage est rationnel (c-à-d. décrit par une expression régulière) alors il est reconnu par un automate fini. La preuve de ce théorème est fondée sur la décomposition d'une expression régulière en ses sous-expressions. Comme suit.
- b. Si l'expression régulière avait été u^* , un automate avec la transition $\{(q_0, u, q_f)\}$ aurait reconnu le même langage, à condition que q_0 soit un état final.
- c. Petite variante, si l'expression régulière avait été u^+ , un automate avec les transitions $\{(q_0, u, q_1), (q_1, u, q_1)\}$ aurait reconnu le même langage, à condition que q_1 soit un état final, mais que q_0 ne le soit pas.
- d. Dans le même esprit, si l'expression avait été $(u \mid v)$, un automate avec les transitions $\{(q_0, u, q_1), (q_0, v, q_1)\}$ aurait reconnu le même langage, à condition que q_1 soit un état final, mais que q_0 ne le soit pas.
- e. Dans le même esprit, si l'expression avait été $(u \bullet v)$, un automate avec les transitions $\{(q_0, u, q_1), (q_1, v, q_2)\}$ aurait reconnu le même langage, à condition que q_2 soit un état final, mais que ni q_0 ni q_1 ne le soit.
- f. Il suffit alors décomposer l'expression régulière selon sa structure (on dit par induction structurelle) et d'appliquer l'un ou l'autre des idées qui précèdent jusqu'à ne plus avoir que des transitions sur des lettres isolées :

$$\begin{aligned}
 \delta &= \{ (q_0, (ab \mid a^+), q_0) \}, & Q_F &= \{ q_0 \} \\
 \delta &= \{ (q_0, ab, q_0), (q_0, a^+, q_0) \}, & Q_F &= \{ q_0 \} \\
 \delta &= \{ (q_0, a, q_1), (q_1, b, q_0), (q_0, a^+, q_0) \}, & Q_F &= \{ q_0 \} \\
 \delta &= \{ (q_0, a, q_1), (q_1, b, q_0), (q_0, a, q_2), (q_2, a, q_0) \}, & Q_F &= \{ q_0 \}
 \end{aligned}$$

Questions :

- a. Comparer avec l'évaluation d'une expression arithmétique.
- b. Comparer avec les lois de la dérivation d'une fonction.

Remarques :

- Cette opération est en général plus compliquée que ces exemples ne le laisse voir. On ne rentrera pas dans ces détails, car on ne fait presque jamais cette opération à la main.
- Cette opération est effectuée de façon routinière et automatique par les logiciels d'analyse de texte par des expressions régulières.
- Cette traduction est la clé d'une démarche où les humains utilisent les expressions régulières, car c'est plus facile pour eux, et des machines utilisent les automates finis, car ce sont en fait des algorithmes.

Exercice 6 — Lemme de l'étoile, ou lemme de pompage

Si $\mathcal{L}$ est rationnel

alors

$\exists N_{\mathcal{L}} . \forall m \in \mathcal{L} .$

si longueur(m) $\geq N_{\mathcal{L}}$

alors

$\exists x, y, z .$

$m = x \cdot y \cdot z$

et

$0 < \text{longueur}(x \cdot y) \leq N_{\mathcal{L}}$

et

$\forall n . x \cdot y^n \cdot z \in \mathcal{L}$

- Que permet de démontrer le lemme de l'étoile ?

Réflexions :

- Ce lemme est de la forme $A \Rightarrow B$. A : être rationnel, B vérifier une propriété existentielle, $\exists N_{\mathcal{L}} . \text{blabla}$.
- C'est équivalent à $\neg B \Rightarrow \neg A$ (contraposition). $\neg B$ ne pas vérifier la propriété existentielle, $\forall N_{\mathcal{L}} . \neg \text{blabla}$, $\neg A$ ne pas être rationnel.
- « *blabla* » peut se paraphraser en : tous les mots suffisamment longs (c-à-d. « $\forall m \in \mathcal{L} . \text{si longueur}(m) \geq N_{\mathcal{L}}$ »), peuvent être décomposés en 3 tronçons (c-à-d. « alors $\exists x, y, z . m = x \cdot y \cdot z$ »), tels que les deux premiers ne sont pas trop longs (c-à-d. « $0 < \text{longueur}(x \cdot y) \leq N_{\mathcal{L}}$ ») et celui du milieu peut être répété autant de fois qu'on veut sans faire sortir le mot du langage (c-à-d. « $\forall n . x \cdot y^n \cdot z \in \mathcal{L}$ »).
- Et « $\neg \text{blabla}$ » se lira donc : il existe toujours un mot suffisamment long, qui ne peut être décomposé en 3 tronçons, tels que les deux premiers ne sont pas trop longs et celui du milieu peut être répété autant de fois qu'on veut sans faire sortir le mot du langage.
- Et donc « $\neg \text{blabla}$ » peut se prouver en imaginant une famille de contre-exemples. Ce qui souvent plus facile que prouver une loi générale.

- f. Conclusion : le lemme de pompage permet de prouver assez facilement qu'un langage n'est pas rationnel.

Questions :

- a. Comment prouver l'équivalence de la contraposition ?
- b. Que se passe-t-il si $\mathcal{L}$ est fini ?

Remarques :

- a. Le lemme de pompage révèle une propriété inattendue qui n'était pas du tout visible dans la définition de « être rationnel ».
 - b. C'est donc un beau théorème ! C'est pourquoi on l'enseigne.
2. Expliquez ce théorème en termes d'automates finis.

Réflexions :

- a. Un automate fini a un nombre fini d'états reliés par un nombre fini de transitions (c'est la définition).
- b. On peut voir cela comme un graphe orienté où les nœuds sont les états et les arcs sont les transitions (c'est la représentation graphique d'un automate). Les arcs sont étiquetés par la lettre qui entre dans la définition de la transition qui correspond.
- c. Comme dans tout graphe, on peut imaginer des chemins qui vont d'un état vers un autre, et on peut définir leur longueur, c-à-d. le nombre d'arcs qu'ils empruntent.
- d. Il y a forcément un nombre fini de chemins qui ne passent pas deux fois par le même état, et donc forcément un plus long des chemins finis. Notons $N_{\mathcal{A}}$ cette longueur.
- e. Vérifier si un mot est accepté revient alors à chercher dans le graphe un chemin qui conduit de l'état initial à un état final et où chaque arc emprunté par le chemin est étiqueté par une lettre du mot, les lettres étant lues dans l'ordre où elles composent le mot.
- f. Dans ces conditions, à un mot accepté m (c-à-d. qui appartient au langage reconnu par l'automate) et dont la longueur est supérieure à $N_{\mathcal{A}}$ doit forcément correspondre un chemin qui passe plusieurs fois par le même état.
- g. Le chemin qui accepte m a donc la forme $q_0 \alpha s \beta s \gamma q_f$. Mais alors, $q_0 \alpha s \gamma q_f$ est aussi un chemin qui relie l'état initial à un état final, $q_0 \alpha s \beta s \beta s \gamma q_f$ en est un autre, et $q_0 \alpha s \beta s \beta s \beta s \gamma q_f$ aussi, etc.
- h. Et donc le $N_{\mathcal{L}}$ du lemme de pompage est devenu le $N_{\mathcal{A}}$ de l'automate, et répéter un tronçon du milieu revient juste à passer autant de fois qu'on veut dans le chemin qui mène de 's' à 's'.

Questions :

- a. À quoi ressemble un automate qui reconnaît un langage fini ?

- b. Est-ce qu'un tel automate peut accepter des mots qui sont plus longs que $N_{\mathcal{A}}$?
- c. Soit un automate $\mathcal{A}$ qui reconnaît un langage $\mathcal{L}$. À quoi peut bien ressembler un automate qui reconnaît $\mathcal{L}^*$? Même question pour $\mathcal{L}^+$ et $\mathcal{L}^?$.

Remarques :

- a. Les automates sont les correspondants matériels des expressions régulières. On peut aisément les représenter dans des programmes, les parcourir, et donc décider si un mot est accepté ou non.
- b. Les langages rationnels sont définis par les expressions rationnelles. Ils sont reconnus par les automates finis. Ils sont caractérisés par le lemme de l'étoile. Qu'il soit possible d'en parler de façons aussi différentes peut être vu comme le signe de la richesse de ce concept.